

Computer Architecture

(Instructions: Part II)

Dept. Applied AI, SeoulTech

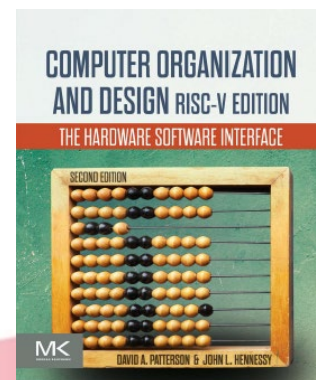
Beomseok Oh

Contents

- Procedure
- A C sort example
- Communicating with people & Wide immediate and address

Procedure

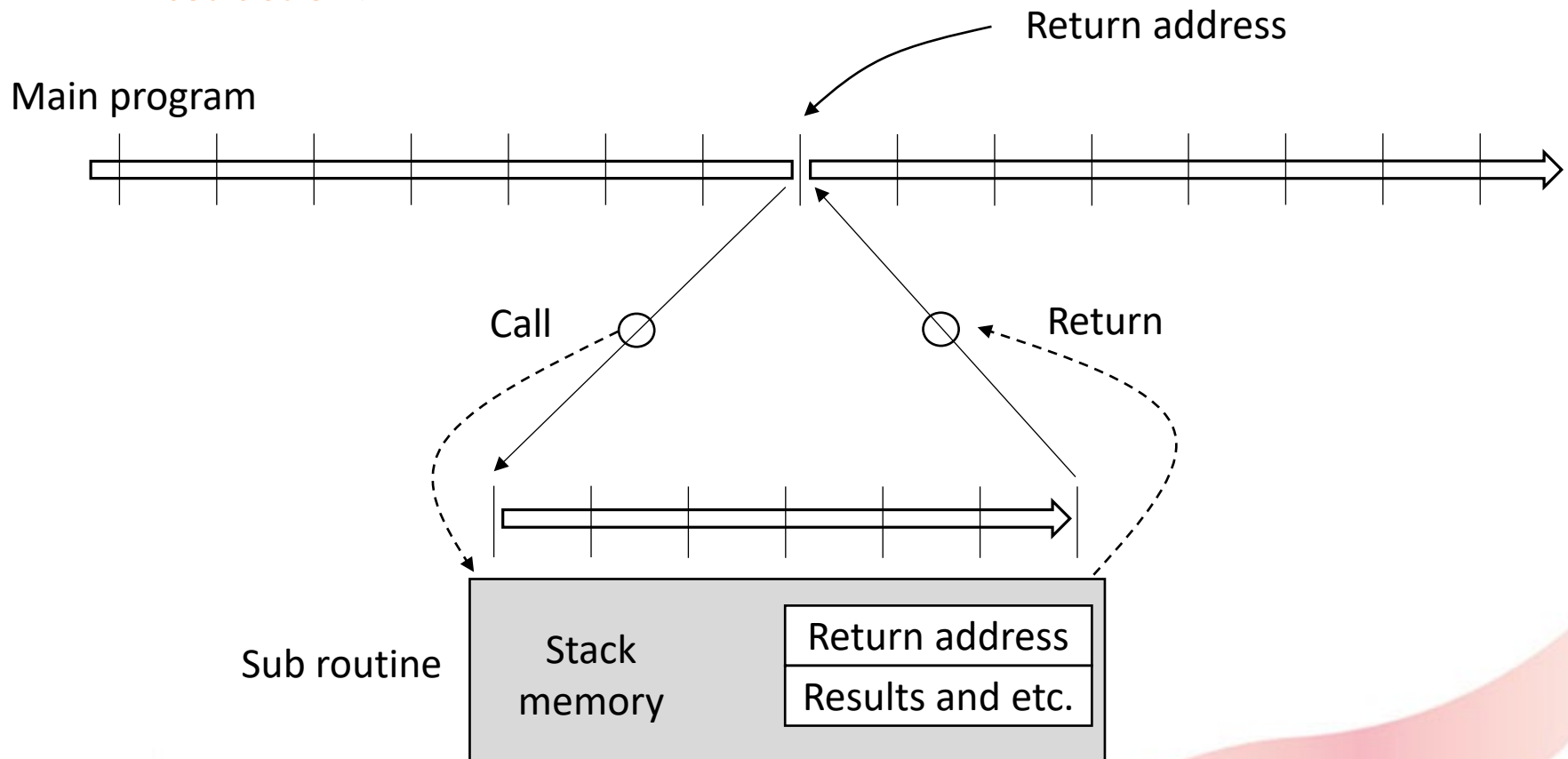
(ch. 2.8, COD book)



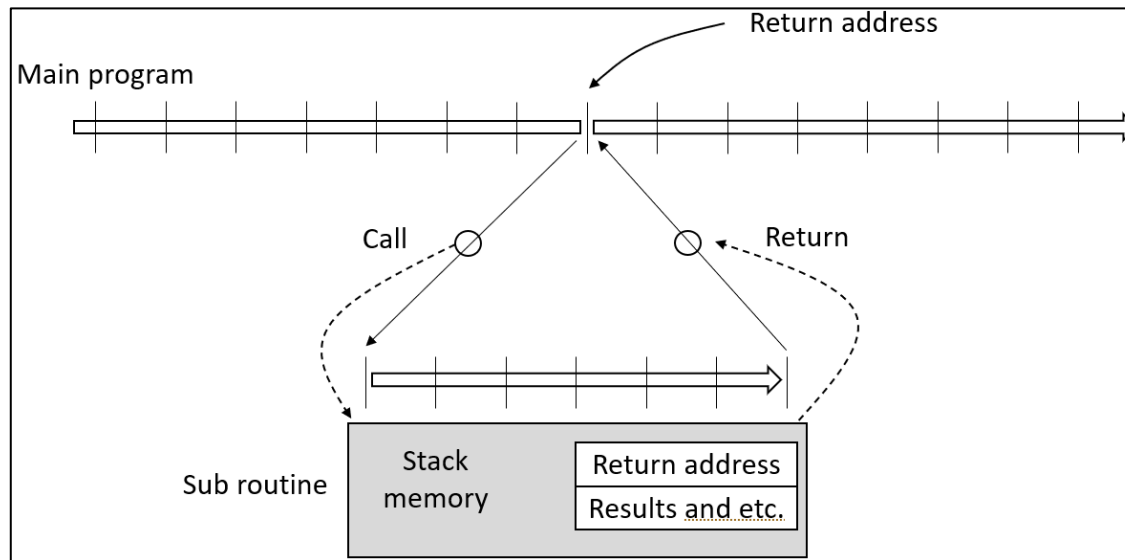
Procedure

- Procedure (or function)

- A stored **subroutine** that performs a specific task with parameters provided
- Easier to understand & allow codes to be reused
- **Abstraction!**



Procedure call



■ Steps required

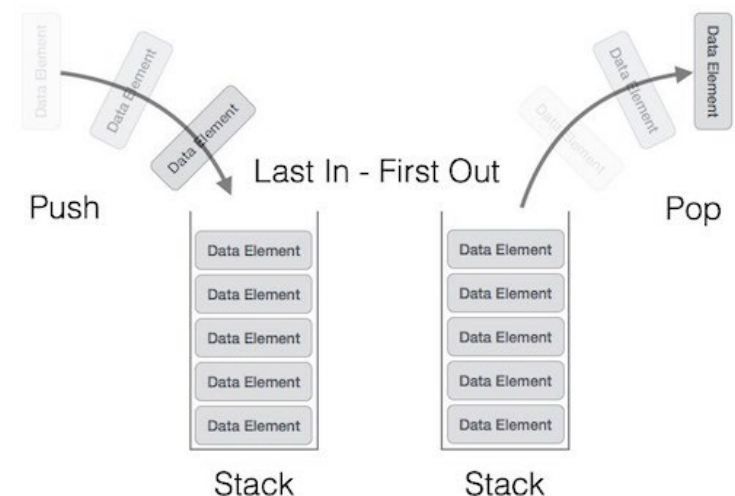
1. Put arguments in a place (registers $x10$ to $x17$) the procedure can access
2. Transfer control to the procedure
3. Acquire storage resource needed for the procedure
4. Perform the desired task of the procedure
5. Put the results in place (one of $x10$ to $x17$) where the caller can access
6. Restore any registers the procedure used; release local storage
7. Return control to the point of origin (address in $x1$)

RISC-V procedure call

- RISC-V register procedure call convention
 - `x10` to `x17`: **eight parameter register** in which to pass parameters or return values
 - `x1`: **one return address register** to return to the point of origin
- Procedure call: jump and link
 - `procedureLabel()` (in C)
 - `jal x1, procedureLabel` (in RISC-V)
 - Saves `PC+4` in `x1`
- Procedure return: jump and link return
 - `return` (in C)
 - `jalr x0, 0(x1)` (in RISC-V)
 - Jumps to `0 + address in x1`
 - Note that `x0` is hard-wired to zero

RISC-V procedure call – Cont'd

- Need **more registers** for a procedure than the eight arguments registers?
 - We can use other registers, but should be careful
 - Since we must cover our tracks after our mission is complete, **any registers** needed by the caller **must be restored** to the values that they contained **before** the procedure call was invoked
- **Spilling registers to memory** using a data structure, **stack**!
 - Push: placing data onto the stack
 - Pop: removing data from the stack
 - **Stack pointer** (sp , register $x2$)
 - Stack “grows” from **higher addresses to lower addresses** meaning that pushing values onto the stack by subtracting from the sp



RISC-V procedure call – Cont'd

- Consider the following example:

```
int leaf_example (int g, int h, int i, int j) {  
    int f;  
    f = (g + h) - (i + j);  
    return f;  
}
```

- Assume that the parameter variables `g`, `h`, `i` and `j` correspond to the argument registers `x10`, `x11`, `x12`, and `x13`, respectively, and `f` corresponds to `x20`
- The compiled program starts with the label `leaf_example`:
- For the statement `f = (g + h) - (i + j)`, we need **two more registers** `x5` and `x6`
- It means that we need to **save three registers** `x20`, `x5` and `x6` onto stack

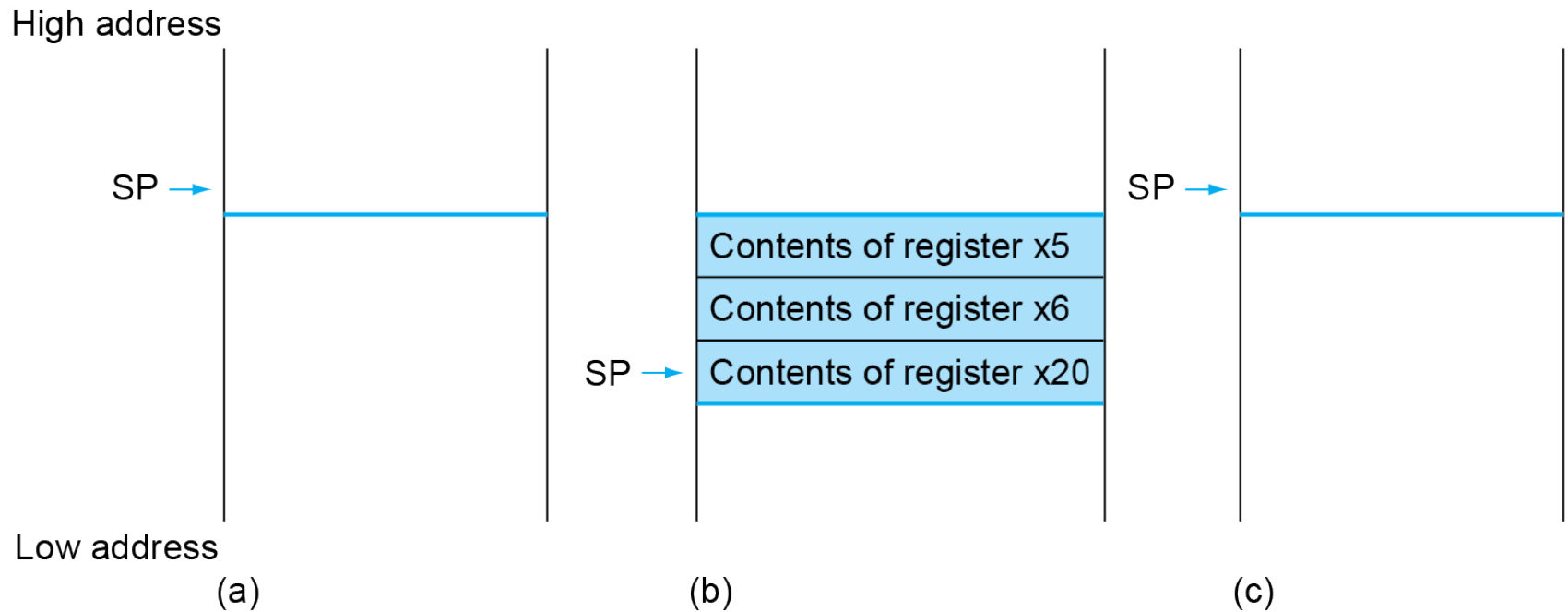
RISC-V procedure call – (

```
int leaf_example (int g, int h, int i, int j) {  
    int f;  
    f = (g + h) - (i + j);  
    return f;  
}
```

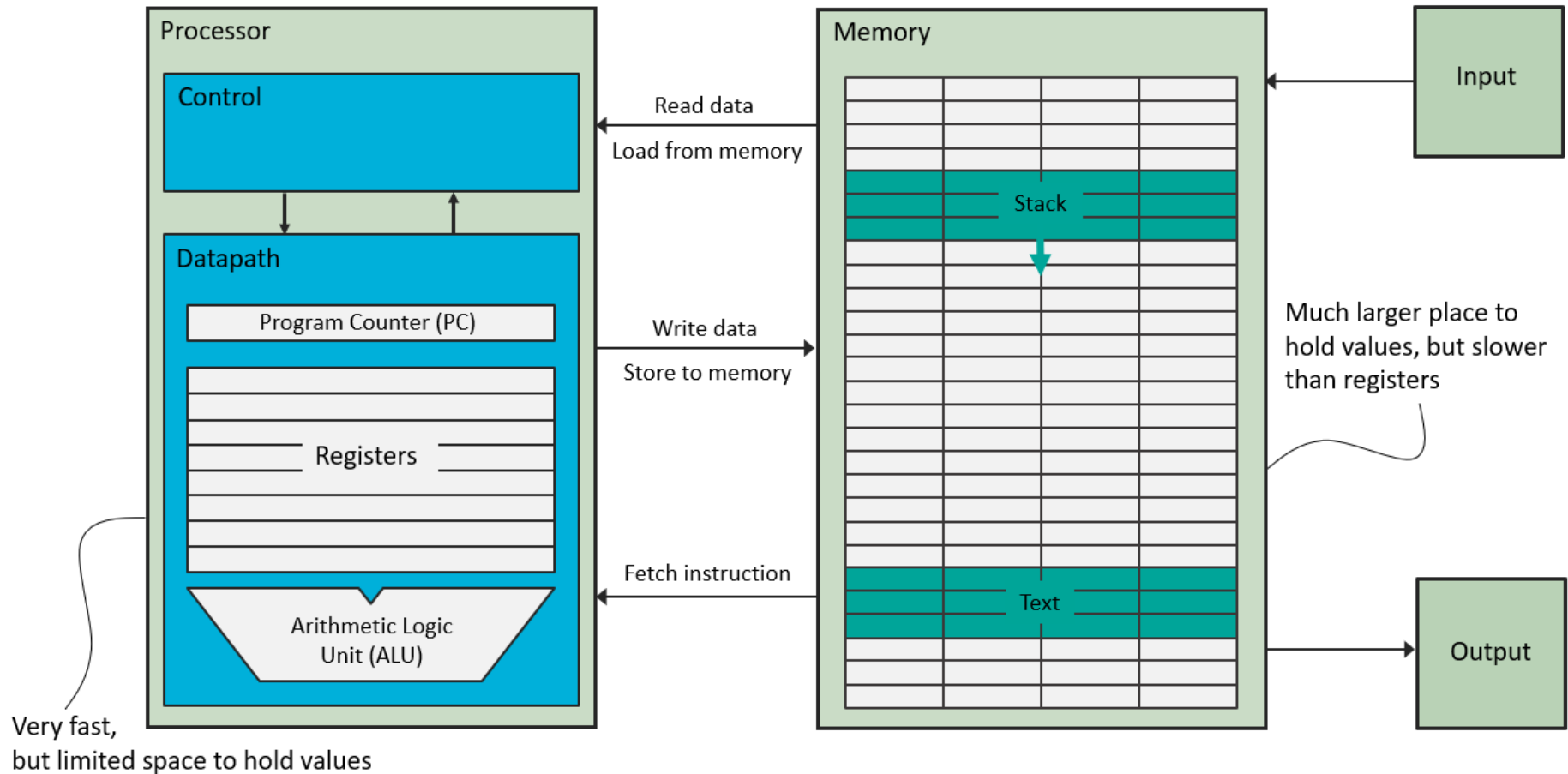
leaf_example:

```
addi sp, sp, -12    # adjust stack to make room for 3 items  
sw    x5, 8(sp)     # save register x5 for use afterwards  
sw    x6, 4(sp)     # save register x6 for use afterwards  
sw    x20, 0(sp)    # save register x20 for use afterwards  
  
add    x5, x10, x11 # register x5 contain g + h  
add    x6, x12, x13 # register x6 contain i + j  
sub    x20, x5, x6  # f = x5 - x6 that is (g + h) - (i + j)  
  
addi x10, x20, 0    # returns f (x10 = x20 + 0)  
  
lw    x20, 0(sp)    # restore register x20 for caller  
lw    x6, 4(sp)     # restore register x6 for caller  
lw    x5, 8(sp)     # restore register x5 for caller  
addi sp, sp, 12     # adjust stack to delete 3 items  
  
jalr x0, 0(x1)      # branch back to calling routine
```

Local Data on the Stack



Local Data on the Stack – Cont'd



RISC-V registers

- RISC-V convention to **reduces register spilling**
 - $x1$ (ra) : **Return address register** to return to the point of origin
 - $x2$ (sp) : **Stack pointer register** to point out the current address of stack
 - $x3$ (gp) : **Global pointer register** to across procedures
 - $x10-x17$ ($a0-a7$) : **Augment registers** in which to pass parameters or return values
 - $x5-x7$ ($t0-t2$) and $x28-x31$ ($t3-t6$) : Temporary registers that are **not preserved by the callee** (called procedure) on a procedure call
 - $x8-x9$ ($s0-s1$) and $x18-x27$ ($s2-s11$) : Saved registers that **must be preserved** on a procedure call (if used, the callee saves and restores them)
- In the previous example, $x5$ and $x6$ are used since the caller does not expect to have those values
 - We **can drop two stores and two loads** from the codes

Nested procedure

- Leaf procedure
 - Procedures that do not call others
- Non-leaf procedure
 - Procedures that call other procedures
 - a.k.a. nested procedure
- For nested calls, **caller needs to save on the stack:**
 - Its return address
 - Any arguments and temporaries needed after the call
- Restore from the stack after the call

Nested procedure – Cont'd

- Consider an example
 - Main program calls procedure A (`jal x1, A`) with an argument 3 (in `x10`)
 - Suppose A calls procedure B (`jal x1, B`) with an argument 7 (in `x10`)
 - There are **conflicts** over the use of registers `x1` and `x10`
 - What should we do?
- A possible solution is to **push all the other registers** that must be preserved **on the stack**
 - **Caller pushes** any argument registers (`x10–x17`) or temporary registers (`x5–x7` and `x28–x31`)
 - **Callee pushes** the return address register `x1` and any saved registers (`x8–x9` and `x18–x27`) used by the callee
 - The **`sp` is adjusted** to account for the number of registers placed on the stack

Non-Leaf procedure example

- Consider the following C code:

```
int fact (int n)
{
    if (n < 1) return 1;
    else return (n * fact(n - 1));
}
```

- Assume that the parameter variable `n` corresponds `x10`
- What is the compiled RISC-V assembly code?

Non-leaf procedure example

fact:

```
addi sp, sp, -8      # adjust stack to make room for 2 items
sw    x1, 4(sp)      # save the return address
sw    x10, 0(sp)     # save the argument n
addi  x5, x10, -1    # x5 = n - 1
bge   x5, x0, else   # if (n - 1) >= 0, go to else
addi  x10, x0, 1     # return 1
addi  sp, sp, 8      # pop 2 items off stack
jalr  x0, 0(x1)      # return to caller
```

else:

```
addi  x10, x10, -1   # n >= 1: argument gets (n - 1)
jal   x1, fact       # call fact with (n - 1)
addi  x6, x10, 0     # return from jal: x6 has factorial(n - 1)

lw    x10, 0(sp)     # restore saved argument n
lw    x1, 4(sp)      # restore the return address
addi  sp, sp, 8      # adjust stack point to pop 2 items

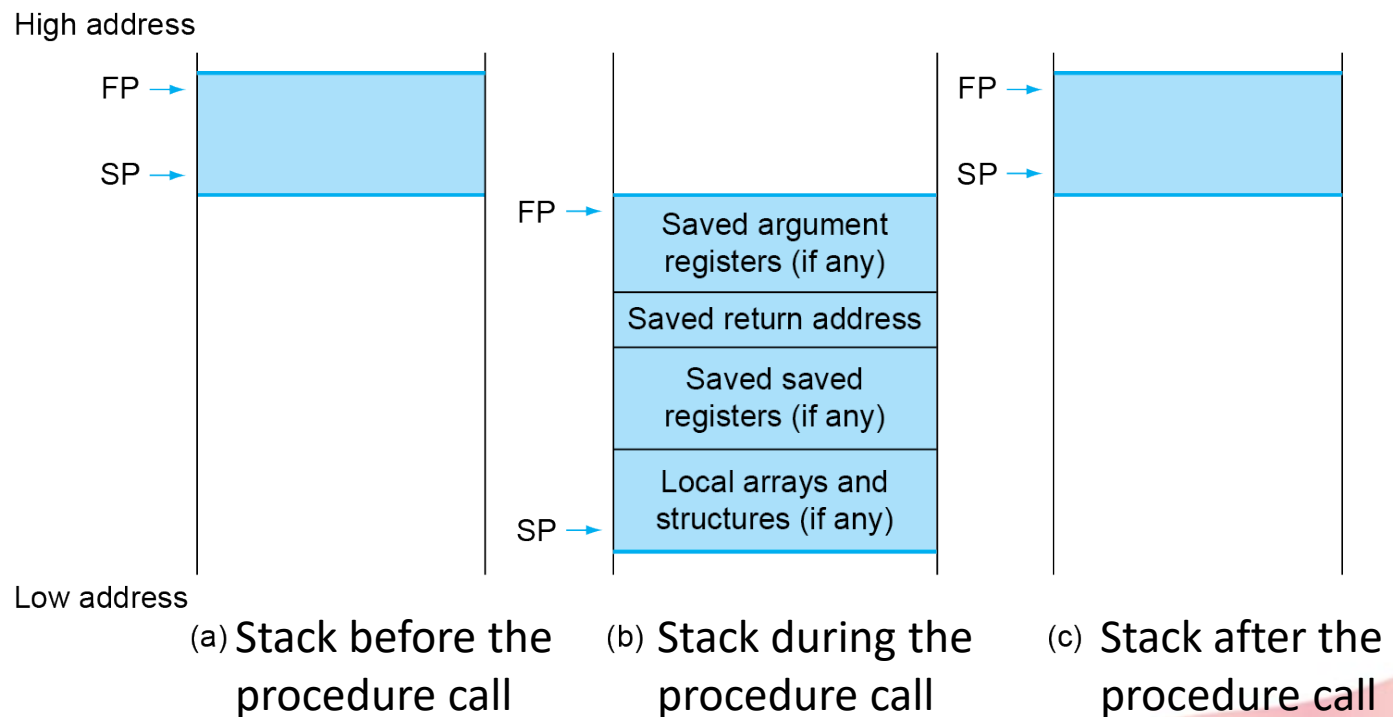
mul   x10, x10, x6    # return n * factorial(n - 1)
jalr  x0, 0(x1)      # return to the caller
```


Automatic and static variables

- Interpretation of C variables depends both on its type and storage class
 - Type: int, char etc.
 - Class: automatic and static
- Automatic variables
 - Local to a procedure
 - Discarded when the procedure exits
- Static variables
 - Exists across exits from and entries to procedures
 - Defined using static keyword or outside any function
- To simplify access to static data, some RISC-V compilers reserve $\times 3$ for use as the global pointer

Frame pointer

- The stack is also used to **store variables that are local** to the procedure but **do not fit in registers**, such as local arrays or structures
- The **segment of the stack** containing a procedure's saved registers and local variables is called a **procedure frame** (or stack frame)



Frame pointer – Cont'd

- A stack pointer **might change** during the procedure, and so references to a local variable in memory **might have different offsets** depending on where they are in the procedure, making the procedure harder to understand
- Some RISC-V compilers use a **frame pointer** register $x8$ ($s0$ or fp) to point to **the first word of the frame** of a procedure
- Compared to a stack pointer, a frame pointer **offers a stable base register** within a procedure for local memory-references
- What if we have more than 8 parameters for a procedure call?
 - The RISC-V convention is to place the first 8 parameters in registers $x10$ through $x17$, and the remaining on the stack **just above the fp**

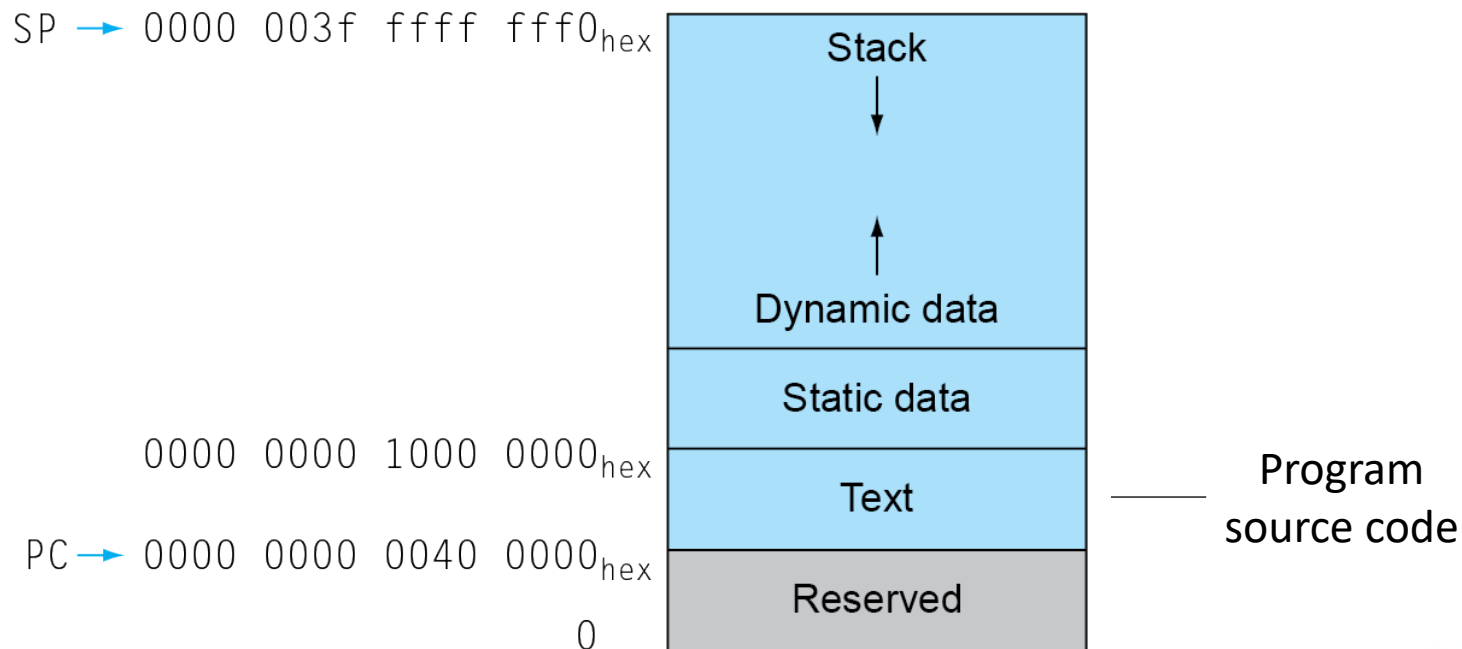
RISC-V register summary

Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5–7	t0–2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10–11	a0–1	Function arguments/return values	Caller
x12–17	a2–7	Function arguments	Caller
x18–27	s2–11	Saved registers	Callee
x28–31	t3–6	Temporaries	Caller

Application binary interface

Memory Layout

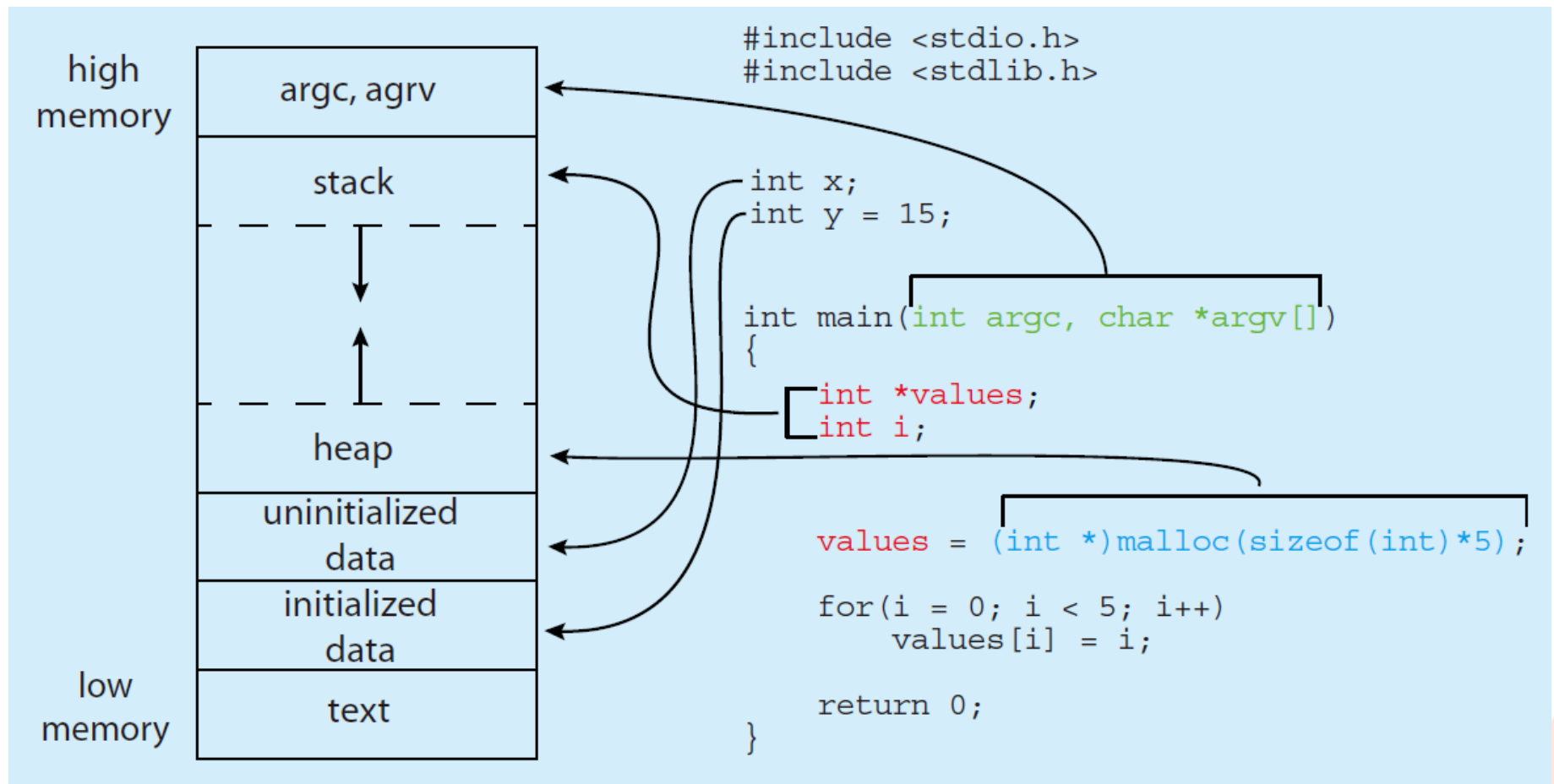
- C program needs space in memory for **static (global) variables** and **dynamic data structures** (e.g., `malloc` in C, `new` in Java)
 - We know that stack is used for **automatic variables**
 - Static and dynamic data structures? **Heap** structure!



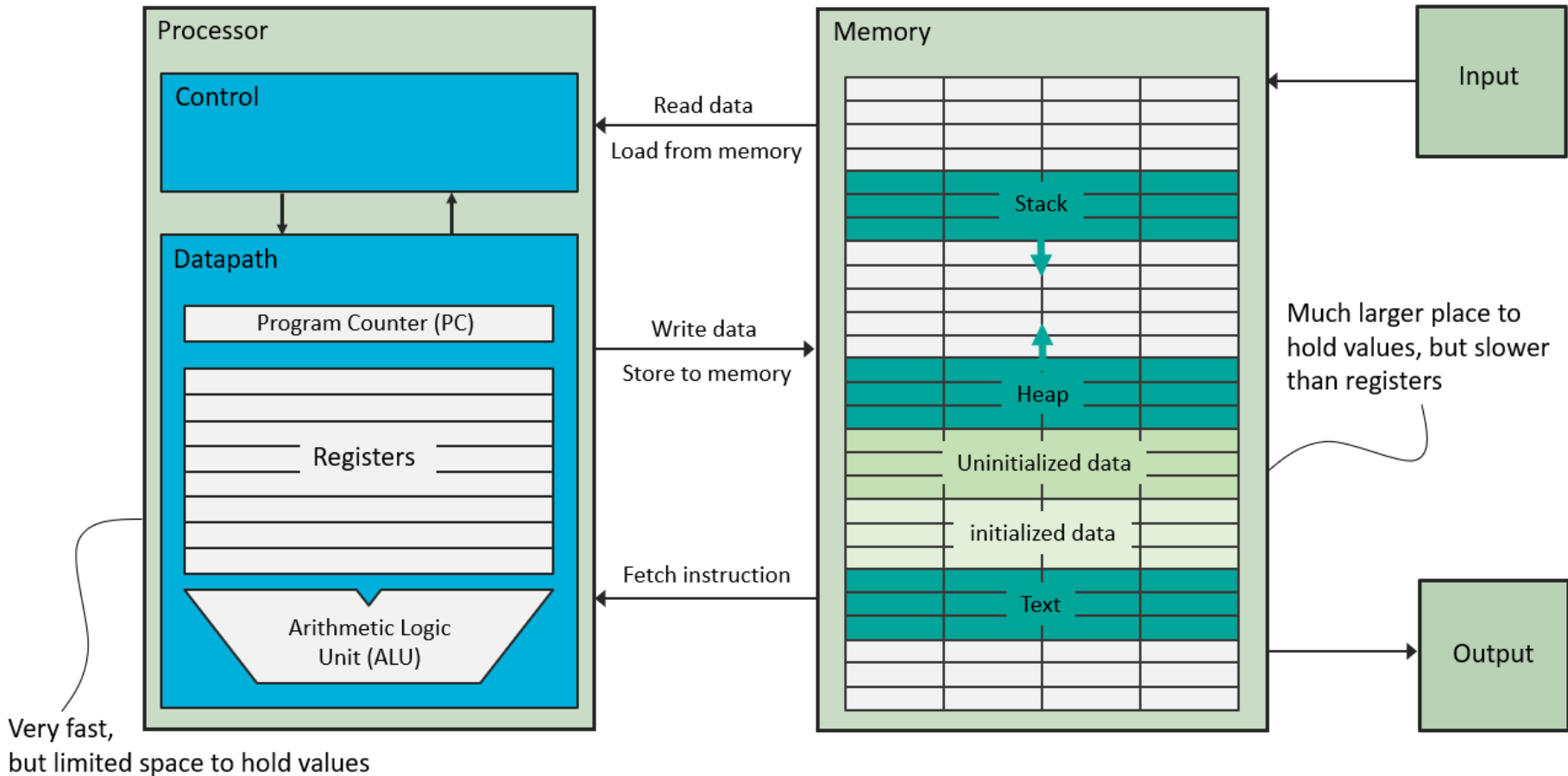
Memory Layout – Cont'd

- Text segment
 - The text segment contains binary code of instructions
- (static) data segment
 - contains **constants** and **static variables** that never change
 - separated into two places:
 - **Initialized data segment** for initialized static variables
 - **Uninitialized data segment** for uninitialized static variables
- Heap segment
 - the segment where **dynamic memory allocation** takes place
 - begins at the end of the uninitialized data segment and **grows to larger addresses** from there
 - managed by `malloc`, `realloc`, and `free`, which may use the `brk` and `sbrk` system calls to adjust its size
- Stack segment
 - is the segment where **local variables** and **some values in registers** takes place
 - **grows to lower addresses**

Memory Layout – Cont'd

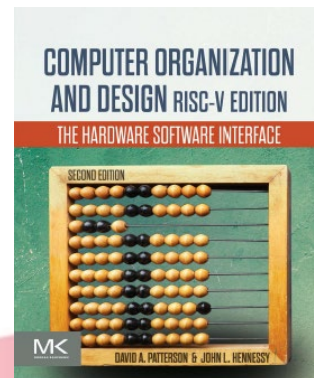


Memory Layout – Cont'd



A C sort example

(ch. 2.13, COD book)



Insertion sort in C

```
void sort(int v[], int n) {  
    for (int i = 1; i < n; ++i) {  
        temp = v[i];  
        for (int j = i - 1; j >= 0 && v[j] >= temp; --j) {  
            v[j+1] = v[j];  
        }  
        v[j+1] = temp;  
    }  
}
```

Insertion sort in C – Cont'd

- Step 1: Prologue (Adjusting stack pointer and saving registers)

Sort:

```
addi sp, sp, -12    # make room on stack for 3 registers
sw    ra, 8(sp)      # save return address register
sw    s1, 4(sp)      # save saved register s1
sw    s2, 0(sp)      # save saved register s2
```

- In this example, we only use temporary registers ($t0-t6$) and saved register ($s1-s2$)
 - caller is obligated to save temporary registers ($t0-t6$)
 - callee is obligated to saved register ($s1-s2$)

Register	ABI Name
x0	zero
x1	ra
x2	sp
x3	gp
x4	tp
x5-7	t0-2
x8	s0/fp
x9	s1
x10-11	a0-1
x12-17	a2-7
x18-27	s2-11
x28-31	t3-6

- x1 (ra): **Return address register** to return to the point of origin
- x2 (sp): **Stack pointer register** to point out the current address of stack
- x3 (gp): **Global pointer register** to across procedures
- x10-x17 (a0-a7): **Augment registers** in which to pass parameters or return values
- x5-x7 (t0-t2) and x28-x31 (t3-t6): Temporary registers that are **not preserved by the callee** (called procedure) on a procedure call
- x8-x9 (s0-s1) and x18-x27 (s2-s11): Saved registers that **must be preserved** on a procedure call (if used, the callee saves and restores them)

Insertion sort in C – Cont'd

■ Step 2: Move parameter

Sort:

```
addi sp, sp, -12    # make room on stack for 3 registers
sw     ra, 8(sp)     # save return address register
sw     s1, 4(sp)     # save saved register s1
sw     s2, 0(sp)     # save saved register s2
addi t0, a0, 0       # copy parameters a0 into t0 (int v[])
addi t1, a1, 0       # copy parameters a1 into t1 (int n)
```

```
void sort(int v[], int n) {
    for (int i = 1; i < n; ++i) {
        temp = v[i];
        for (int j = i - 1; j >= 0 && v[j] >= temp; --j) {
            v[j+1] = v[j];
        }
        v[j+1] = temp;
    }
}
```

Insertion sort in C – Cont'd

```
void sort(int v[], int n) {  
    for (int i = 1; i < n; ++i) {  
        temp = v[i];  
        for (int j = i - 1; j >= 0 && v[j] >= temp; --j) {  
            v[j+1] = v[j];  
        }  
        v[j+1] = temp;  
    }  
}
```

■ Step 3: Outer loop 1

```
sw    s1, 4(sp)    # save saved register s1  
sw    s2, 0(sp)    # save saved register s2  
addi  t0, a0, 0    # copy parameters a0 into t0 (int v[])  
addi  t1, a1, 0    # copy parameters a1 into t1 (int n)  
addi  t2, x0, 1    # t2 has i=1
```

Loop1:

```
bge   t2, t1, Exit  # if (i >= n) goto Exit  
slli  t4, t2, 2     # t4 has offset i * 4  
add   s1, t0, t4    # s1 has address of v[i]  
lw    t4, 0(s1)     # t4 has v[i]
```

Insertion sort in C – Cont'd

```
void sort(int v[], int n) {  
    for (int i = 1; i < n; ++i) {  
        temp = v[i];  
        for (int j = i - 1; j >= 0 && v[j] >= temp; --j) {  
            v[j+1] = v[j];  
        }  
        v[j+1] = temp;  
    }  
}
```

■ Step 4: Inner loop

```
add    s1, t0, t4    # s1 has address of v[i]  
lw     t4, 0(s1)     # t4 has v[i]  
addi   t3, t2, -1    # t3 has j=i-1
```

Loop2:

```
blt    t3, x0, Loop3 # if (j < 0) goto Loop3  
slli   t5, t3, 2     # t5 has offset j * 4  
add    s2, t0, t5     # s2 has address of v[j]  
lw     t5, 0(s2)     # t5 has v[j]  
blt    t5, t4, Loop3 # if (v[j] < temp) goto Loop3  
sw     t5, 4(s2)     # v[j+1] = v[j]  
addi   t3, t3, -1    # j--  
jal    x0, Loop2     # goto Loop2
```

Insertion sort in C – Cont'd

```
void sort(int v[], int n) {  
    for (int i = 1; i < n; ++i) {  
        temp = v[i];  
        for (int j = i - 1; j >= 0 && v[j] >= temp; --j) {  
            v[j+1] = v[j];  
        }  
        v[j+1] = temp;  
    }  
}
```

■ Step 5: Outer loop 2

```
blt    t5, t4, Loop3    # if (v[j] < temp) goto Loop3  
sw     t5, 4(s2)         # v[j+1] = v[j]  
addi   t3, t3, -1       # j--  
jal    x0, Loop2         # goto Loop2
```

Loop3:

```
slli   t5, t3, 2         # t5 has offset j * 4  
add    s2, t0, t5        # s2 has address of v[j]  
sw     t4, 4(s2)         # v[j+1] = temp  
addi   t2, t2, 1         # i++  
jal    x0, Loop1         # goto Loop2
```

Insertion sort in C – Cont'd

```
void sort(int v[], int n) {  
    for (int i = 1; i < n; ++i) {  
        temp = v[i];  
        for (int j = i - 1; j >= 0 && v[j] >= temp; --j) {  
            v[j+1] = v[j];  
        }  
        v[j+1] = temp;  
    }  
}
```

Loop3:

```
slli t5, t3, 2      # t5 has offset j * 4  
add s2, t0, t5      # s2 has address of v[j]  
sw t4, 4(s2)        # v[j+1] = temp  
addi t2, t2, 1      # i++  
jal x0, Loop1       # goto Loop2
```

Exit:

```
lw s2, 0(sp)        # restore saved register s2  
lw s1, 4(sp)        # restore saved register s1  
lw ra, 8(sp)        # restore return address  
addi sp, sp, 12     # restore stack pointer  
jalr x0, 0(ra)      # return to caller
```



```

void sort(int v[], int n) {
    for (int i = 1; i < n; ++i) {
        temp = v[i];
        for(int j=i-1; j>=0 && v[j]>=temp; --j)
        {
            v[j+1] = v[j];
        }
        v[j+1] = temp;
    }
}

```

Compiler
→

Sort:

```

addi sp, sp, -12    # make room on stack
sw   ra, 8(sp)      # save return address regs.
sw   s1, 4(sp)      # save saved register s1
sw   s2, 0(sp)      # save saved register s2
addi t0, a0, 0      # t0 has *v
addi t1, a1, 0      # t1 has n
addi t2, x0, 1      # t2 has i=1

```

Loop1:

```

bge t2, t1, Exit    # if (i >= n) goto Exit
slli t4, t2, 2      # t4 has offset i * 4
add s1, t0, t4      # s1 has address of v[i]
lw  t4, 0(s1)       # t4 has v[i]
addi t3, t2, -1     # t3 has j=i-1

```

Loop2:

```

blt t3, x0, Loop3   # if (j < 0) goto Loop3
slli t5, t3, 2      # t5 has offset j * 4
add s2, t0, t5      # s2 has address of v[j]
lw  t5, 0(s2)       # t5 has v[j]
blt t5, t4, Loop3   # if (v[j] < temp) goto L3
sw  t5, 4(s2)       # v[j+1] = v[j]
addi t3, t3, -1     # j--
jal x0, Loop2       # goto Loop2

```

Loop3:

```

slli t5, t3, 2      # t5 has offset j * 4
add s2, t0, t5      # s2 has address of v[j]
sw  t4, 4(s2)       # t4 has v[j+1]
addi t2, t2, 1      # i++
jal x0, Loop1       # goto Loop1

```

Exit:

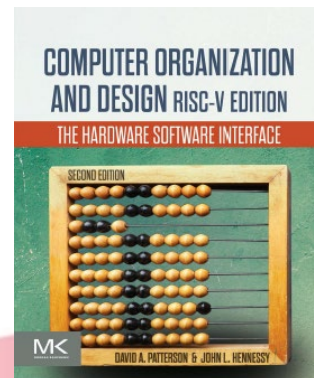
```

lw  s2, 0(sp)      # restore saved register s2
lw  s1, 4(sp)      # restore saved register s1
lw  ra, 8(sp)      # restore return address
addi sp, sp, 12    # restore stack pointer
jalr x0, 0(ra)     # return to caller

```

Communicating with people & Wide immediate and address

(ch. 2.9 and 2.10, COD book)



Character Data

- Byte-encoded character sets
 - ASCII: 128 characters
 - 95 graphic, 33 control
 - Latin-1: 256 characters
 - ASCII, +96 more graphic characters
- Unicode: 32-bit character set
 - Used in Java, C++ wide characters, ...
 - Most of the world's alphabets, plus symbols
 - UTF-8, UTF-16: variable-length encodings
- A series of instructions can extract a byte from a word
- RISC-V also provides **instructions for bytes**

E.g., `lbu x12, 0(x10)`
`sb x12, 0(x11)`

Character Data – Cont'd

- ASCII (American Standard Code for Information Interchange) code

ASCII value	Character	ASCII value	Character	ASCII value	Character	ASCII value	Character	ASCII value	Character	ASCII value	Character
32	space	48	0	64	@	80	P	96	`	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	\$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o	127	DEL

Byte/Halfword/Word Operations

- RISC-V byte/halfword/word load/store
 - Load byte/halfword/word: Sign extend to 64 bits in `rd`
`lb rd, offset(rs1)`
`lh rd, offset(rs1)`
`lw rd, offset(rs1)`
 - Load byte/halfword/word unsigned: Zero extend to 64 bits in `rd`
`lbu rd, offset(rs1)`
`lhu rd, offset(rs1)`
`lwu rd, offset(rs1)`
 - Store byte/halfword/word: Store rightmost 8/16/32 bits
`sb rs2, offset(rs1)`
`sh rs2, offset(rs1)`
`sw rs2, offset(rs1)`
 - Load/store example
`lbu x12, 0(x10)`
`sb x12, 0(x11)`

Representation of string in C

- String is a set of characters with variable numbers
 - In C program, the last position of a string is indicated by a character used to mark the end of the string
 - E.g., "Cal" is represented by the following 4 bytes 67, 97, 108, 0

ASCII value	Character	ASCII value	Character	ASCII value	Character	ASCII value	Character	ASCII value	Character	ASCII value	Character
32	space	48	0	64	@	80	P	96	`	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	\$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o	127	DEL

String Copy Example

- Consider the following

```
void strcpy (char x[], char y[])
{
    size_t i;
    i = 0;
    while ((x[i]=y[i])!='\0')
        i += 1;
}
```

String Copy Example – Cont'd

■ RISC-V code:

strcpy:

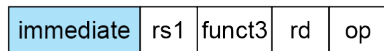
```
    addi sp, sp, -4           # adjust stack for 1 more item
    sw    x19, 0(sp)          # save (push) x19
    add   x19, x0, x0          # i=0+0
L1:  add   x5, x19, x11        # x5 = address of y[i]
     lbu   x6, 0(x5)           # x6 = y[i]
     add   x7, x19, x10        # x7 = address of x[i]
     sb    x6, 0(x7)           # x[i] = y[i]
     beq   x6, x0, L2          # if y[i] == 0 then exit
     addi  x19, x19, 1         # i = i + 1
     jal   x0, L1              # next iteration of loop (go to L1)
L2:  lw    x19, 0(sp)          # restore old (saved) x19
     addi  sp, sp, 4           # pop 1 word off stack
     jalr  x0, 0(x1)           # return
```

```
void strcpy (char x[], char y[])
{
    size_t i;
    i = 0;
    while ((x[i]=y[i])!='\0')
        i += 1;
}
```

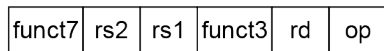

RISC-V Addressing mode summary

Name (Field Size)	Field					Comments	
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits		
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode	Unconditional jump format
U-type	immediate[31:12]				rd	opcode	Upper immediate format

1. Immediate addressing



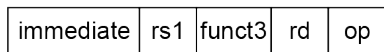
2. Register addressing



Registers

Register

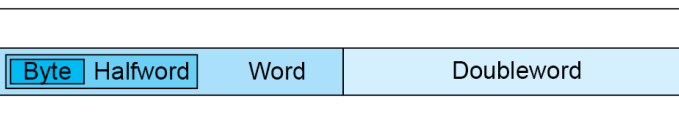
3. Base addressing



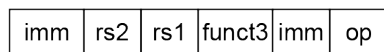
Memory

Register

+



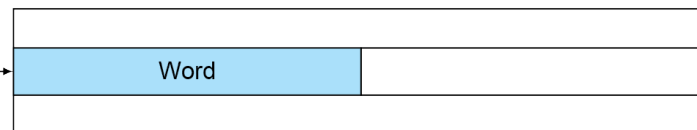
4. PC-relative addressing



Memory

PC

+



Decoding machine code

- Assume that we have the following code to decode

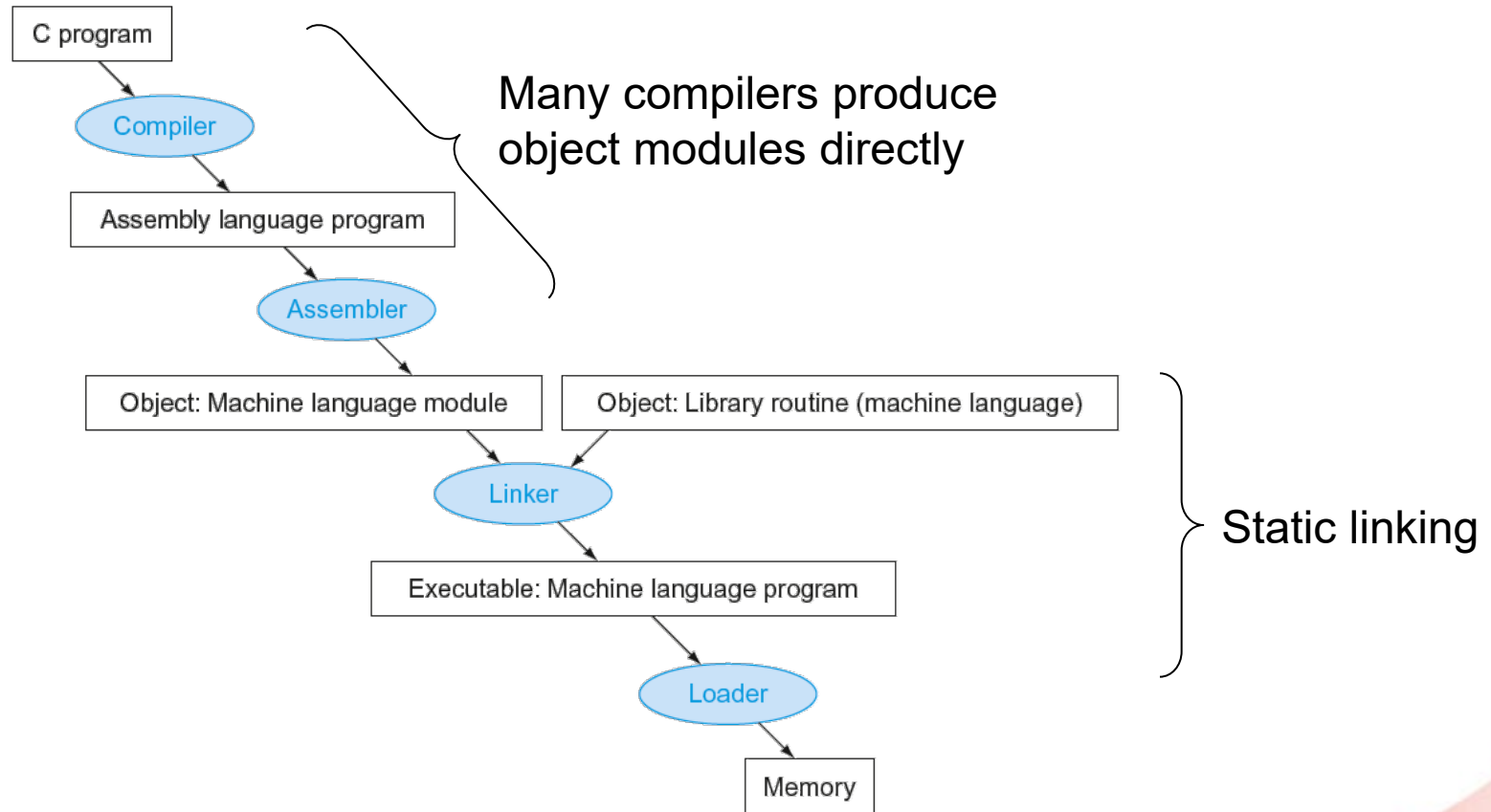
00578833_{hex}

which equals to

0000 0000 0101 0111 1000 1000 0011 0011

Format	Instruction	Opcode	Funct3	Funct6/7
R-type	add	0110011	000	0000000
	sub	0110011	000	0100000
	sll	0110011	001	0000000
	xor	0110011	100	0000000
	srl	0110011	101	0000000
	sra	0110011	101	0000000
	or	0110011	110	0000000
	and	0110011	111	0000000
	lrd	0110011	011	0001000
	scd	0110011	011	0001100
I-type	lb	0000011	000	n.a.
	lh	0000011	001	n.a.
	lw	0000011	010	n.a.
	lbu	0000011	100	n.a.
	lhu	0000011	101	n.a.
	addi	0010011	000	n.a.
	slli	0010011	001	0000000
	xori	0010011	100	n.a.
	srlr	0010011	101	0000000
	srai	0010011	101	0100000
	ori	0010011	110	n.a.
	andi	0010011	111	n.a.
	jalr	1100111	000	n.a.
	sb	0100011	000	n.a.
S-type	sh	0100011	001	n.a.
	sw	0100011	010	n.a.
	sw	0100011	010	n.a.
SB-type	beq	1100111	000	n.a.
	bne	1100111	001	n.a.
	blt	1100111	100	n.a.
	bge	1100111	101	n.a.
	bltu	1100111	110	n.a.
	bgeu	1100111	111	n.a.
U-type	lui	0110111	n.a.	n.a.
UJ-type	jal	1101111	n.a.	n.a.

Translation and Startup



Thank you!